Complex & Intelligent Systems (2024) 10:4713–4729
https://doi.org/10.1007/s40747-024-01402-6

ORIGINAL ARTICLE

Predicting popularity trend in social media networks with multi-layer temporal graph neural networks

Ruidong Jin[1,2] · Xin Liu[2] · Tsuyoshi Murata[1,2]

Received: 15 October 2023 / Accepted: 22 February 2024 / Published online: 2 April 2024
© The Author(s) 2024

Abstract

Predicting what becomes popular on social media is crucial because it helps us understand future topics and public interests based on massive social data. Previous studies mainly focused on picking specific features and checking past statistic numbers, ignoring the hidden impact of messages passing along the complex relationships among different entities. People talk and connect with others on social media; thus, it is essential to consider how information spreads when studying social media networks. This work proposes a multi-layer temporal graph neural network (GNN) framework for predicting what will be popular on social media networks. This framework takes into account the way information spreads among different entities. The proposed method involves multi-layer relations and temporal information within a sequence of social media network snapshots. It learns the temporal representations of target entities in each snapshot and predicts how the popularity of a particular entity will change in future snapshots. The proposed method is evaluated with real-world data across four popularity trend prediction tasks. The experimental results prove that the proposed method performs better than various baselines, including traditional machine learning regression approaches, prior methods for popularity trend prediction, and other GNN models.

Keywords Trend prediction · Social media network · Temporal graph neural network · Heterogeneous graph

Introduction

The proliferation of the internet and the widespread adoption of mobile devices have contributed to the exponential growth in the usage of social media networks, such as X (Twitter), Facebook, YouTube, and Reddit. These platforms collectively boast hundreds of millions of users worldwide. Owing to this vast user base and the copious amounts of data continuously posted and shared, social networks have emerged as invaluable data sources for diverse areas of research, including social recommendation [1], community detection [2], anomaly detection [3], and more. Analyzing and predicting trends within social media networks is an increasingly attractive topic [4]. Trend prediction refers to the process that uses historical trend data and current status data to help interpret the future estimated behavioral pattern. This research provides invaluable insights into the future trajectories of public concerns and interests grounded in the wealth of available social data. From a platform perspective, precise trend prediction holds great significance in enhancing user experiences, elevating service quality, and benefiting various applications.

Trend prediction in social media networks poses a formidable challenge because of its heterogeneity and temporal dynamics. As illustrated in Fig. 1, many social media networks exhibit heterogeneity, featuring various types of entities and multiple types of relations. These relations encompass connections within entities of the same type and connections between entities of different types. Assuming that the trend prediction task is to predict the visit frequency of items in the future, the representations of item entities are acquired only based on the entity attributes. However, it is necessary to consider the impact of information diffusion within entities, such as the interactions and mes-

✉ Xin Liu
xin.liu@aist.go.jp

Ruidong Jin
ruidong.jin@net.c.titech.ac.jp

Tsuyoshi Murata
murata@c.titech.ac.jp

[1] Department of Computer Science, School of Computing, Tokyo Institute of Technology, W8-59 2-12-1 Ookayama, Meguro, Tokyo 152-8552, Japan

[2] Artificial Intelligence Research Center, National Institute of Advanced Industrial Science and Technology, 2-4-7 Aomi, Koto, Tokyo 135-0064, Japan

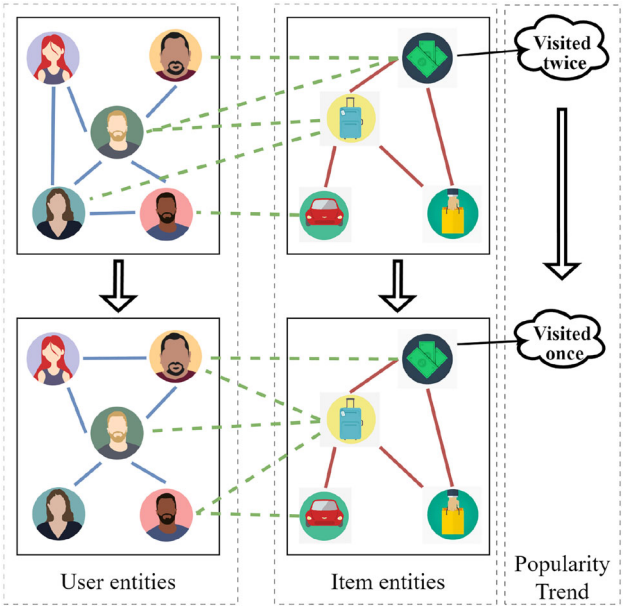


Fig. 1 Example of a social media network consists of users and items. Solid lines denote the interactions between users and items, and dash lines denote the interactions between user and item pairs. Interactions and entity attributes evolve as time goes by. The popularity trend of an item is represented by the visit frequency, which indicates the number of accesses from users (the degree of dashed lines)

sages exchanging within entities. Messages of social media information diffuse through the relations from one entity to another, exerting an impact on their future evolution. Thus, it is necessary to consider the impact of information diffusions within entities. Moreover, it is imperative to capture the temporal dynamics of entities at various timestamps. Social media networks are in a constant state of flux over time, which implies that entity attributes and relations undergo continuous transformations. Temporal dynamics are valuable for interpreting the change pattern of the predicted target.

Based on the analysis above, this work needs to solve the following research questions to address the challenging points mentioned above: RQ1: How to learn the relationships within entities in social media networks? RQ2: How to incorporate the temporal dynamics of entities and relationships into analyzing? RQ3: How to learn the dynamic representations of entities in social media networks? RQ4: how to quantify the degree of future popularity trends and predict it? RQ5: How to test the efficiency of our proposed method?

Several prior works have delved into the task of popularity trend prediction in social networks [5–8]. However, these studies predominantly focus on selecting characteristics and historical statistics within social media networks, often overlooking the substantial impact of information diffusion among entities. Furthermore, most of these prior works employ simplistic machine learning or statistical models (e.g., Support Vector Machine (SVM), Logistic Regression (LR), and more) as encoders and decoders, resulting in suboptimal learning efficiency. Consequently, there exists a pressing need for an advanced framework capable of considering the influence of information diffusion in social media networks while efficiently mastering the representations of target entities.

Graph embedding methods and Graph Neural Networks (GNN) are widely regarded as ideal approaches for modeling the influence of information diffusion in social media networks. Graph embedding methods transform complex graph attributes into low-dimensional embedding vectors while maximally preserving the essential graph structural information. These methods have consistently demonstrated superior performance compared to traditional techniques for modeling graph-structured data [9, 10]. GNNs, on the other hand, belong to the category of deep learning structures designed to perform optimized transformations on all graph attributes while preserving graph topology [11]. GNNs acquire node representations by iteratively aggregating information from neighbor nodes, mirroring the information flow dynamics seen in social media networks. Prior research has also highlighted the adaptability of GNNs in handling temporal graphs [12–15] and heterogeneous graphs [16–19], laying the foundation for learning in the context of social media networks.

In light of the aforementioned considerations, this paper introduces a novel approach employing a multi-layer temporal GNN to address the task of popularity trend prediction in social media networks. The method input is a timed sequence of graph snapshots demonstrating the dynamics of social media networks, and the output is an estimated popularity attribute (e.g., visit frequency, expected revenue, and so on) in the future, indicating the popularity trends of the target entities. The novelty of the proposed method is its unified structure that integrates a multi-layer GNN encoder and a time-sequence unit to effectively learn the temporal node representations by modeling the latent influence of heterogeneous relations and temporal dynamics within social media networks (Answer to RQ1, RQ2, and RQ3). Besides, a Graph Structure Learning (GSL) module is implemented to enhance the quality of input graphs, which is particularly essential when dealing with real-world graphs. Specifically, the evolution of social media networks is characterized as a discrete-time temporal graph comprising a timed sequence of graph snapshots separated by specific time intervals. Within each graph snapshot, node attributes and the intricate multi-layer graph structures are scrutinized to forecast how a specific target node status will evolve in subsequent graph snapshots (Answer to RQ4).

The proposed method is evaluated on four real-world popularity trend prediction tasks using benchmark data sets (Answer to RQ5). The experimental results consistently demonstrate the high efficiency of the proposed approach.

The approach consistently outperforms various baseline methods, including traditional linear regression algorithms, time-sequence models, prior popularity trend prediction methods, and recently introduced heterogeneous GNN methods.

In this paper, we make several significant contributions to the field of trend prediction tasks in social media networks:

- We introduce an advanced multi-layer temporal GNN framework designed to learn entity representations and predict trends within complex real-world social networks. It addresses the crucial task of forecasting trajectories of public concerns and interests based on extensive real-world data.
- We address the limitations of previous research regarding popularity trend predictions, which often neglect the latent influence of complex relationships among entities. The proposed approach presents its novelty in considering the latent influence of information diffusion in social networks while efficiently mastering the representations of target entities, resulting in a substantial performance enhancement.
- We demonstrate the effectiveness of the proposed approach through experiments conducted on real-world social network data sets, demonstrating the feasibility of the proposed method in real-world situations. The proposed method consistently outperforms various baseline methods, including linear regression approaches, time-series models, previous popularity trend prediction methods, and recently proposed heterogeneous GNN methods.

The remainder of the paper is organized as follows. In "Related work" Section summarizes the recent literature on graph embedding learning and trend prediction. In "Problem setup" Section introduces the problem statement of the popularity trend prediction task in social media networks. In "Methodology" Section presents the proposed multi-layer temporal GNN framework and its key components. In "Experiments" Section reports experimental results, evaluation, and additional discussions regarding parameter sensitivity, ablation study, and statistical test. In "Discussion and conclusion" Section concludes this research in the last.

Related work

Graph embedding learning

Graph embedding learning has achieved notable success in various domains [9, 10]. The techniques for generating embedding vectors on graphs have gained widespread recognition, particularly for downstream graph-related tasks such as node classification [20], link prediction [21], and graph classification [22]. The primary challenge in graph learning lies in discovering effective methods to encode graph structures, encompassing nodes and edges, into low-dimensional hidden embedding vectors while maximizing the preservation of essential graph structural information.

GNNs have emerged as a robust deep learning framework for graph embedding learning [11]. Among these, the graph convolutional network (GCN) [20] stands out as one of the most widely adopted models. GCN efficiently aggregates neighbor node information through graph convolutional layers. To further enhance GNN performance, numerous researchers have introduced advanced GNN models building upon the GCN architecture. For example, GraphSAGE extends GCN into inductive learning that can handle unknown graph nodes [23]. Graph attention networks employ attention mechanisms that assign importance weights to different neighbor nodes [24]. Graph autoencoder and variational graph autoencoder are GCN-based encoder–decoder models that can handle unsupervised learning tasks [25]. Moreover, ML-GCN and ML-GAT extend the original GCN and GAT to multi-layer networks, which can capture more complex relations among nodes within large-scale graphs [26].

Learning on temporal graphs presents considerably greater complexity compared to static graphs. Numerous studies have focused on discrete-time temporal graphs, which consist of a timed sequence of graph snapshots [12–14, 27]. Existing static graph methods can be directly applied to each individual graph snapshot. In parallel, substantial research efforts have been dedicated to graph embedding learning for heterogeneous graphs [16, 19]. Heterogeneous graphs encompass nodes and edges of multiple types. HetGNN combines heterogeneous structural information and node attributes. It performs excellently in graphs with multiple types of nodes and edges [18]. HAN is a heterogeneous GNN based on the hierarchical attention mechanism, including node-level and semantic-level attention [17]. It fully considers the importance of node neighbors and different meta-paths. This research endeavors to develop a unified framework that integrates the characteristics of discrete-time temporal graphs and multi-layer heterogeneous graphs.

Trends prediction tasks in real-world data

Time-aware trend prediction tasks have recently attracted much attention in both academia and industry [4–6]. Existing works about trend prediction can be categorized into two primary patterns: The first is to predict the growth and decline of entities based on past characteristics and early stage patterns. Yang et al. found that the temporal patterns reveal how the content popularity fluctuates during the post-propagation [28]. The second is to predict the value of specific target attributes based on temporal attributes or dynamic

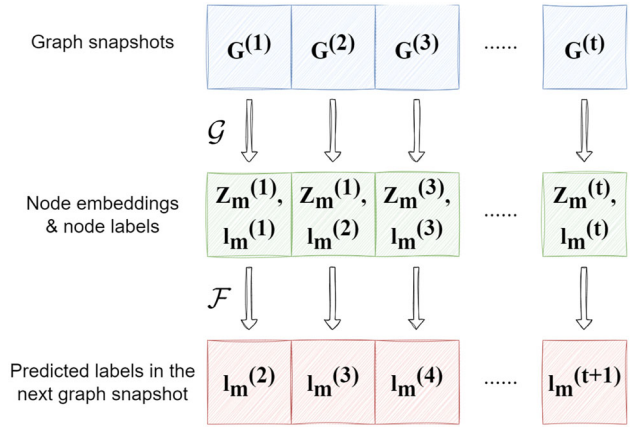


Fig. 2 Problem setup of the popularity trend prediction in social networks. The social networks are presented as a sequence of graph snapshots $\mathbf{G} = \{\mathbf{G}^{(1)} - \mathbf{G}^{(t)}\}$. The proposed method first learns the node embeddings $\mathbf{Z_m}^{(1)} - \mathbf{Z_m}^{(t)}$ of the target node type m by an encoder $\mathcal{G}$, then predicts the node labels $\mathbf{l_m}^{(2)} - \mathbf{l_m}^{(t+1)}$ in the upcoming graph snapshots by a decoder $\mathcal{F}$

signals. Zhao et al. incorporated human reaction time as temporal variables in self-exciting point processes [29]. In addition, some hybrid methods between metaheuristics and machine learning lead to a novel research field, which successfully combines machine learning and swarm intelligence approaches and proved capable of obtaining outstanding results in different trend prediction areas [30–32].

Employing graph-based methods to tackle trend prediction tasks is a relatively recent research direction. He et al. designed a time-aware bipartite graph for estimating the future popularity of items [33]. Cao et al. developed a coupled GNN model to solve the popularity trend prediction task [7]. Li et al. adopted a graph kernel approach to predict the node popularity within a cascade graph sequence [8]. Hou et al. proposed a spatial–temporal multi-graph convolutional network for casualty prediction of terrorist attacks [34]. Yang et al. predicted traffic propagation flow in urban road networks with a multi-graph convolutional network model [35]. However, these methods did not account for multiple relations among various types of entities, limiting their applicability in complex real-world scenarios.

Problem setup

The temporal social network data are structured as a sequence of graph snapshots, each separated by a specific time interval. Many social networks exhibit heterogeneity, featuring various types of entities and multiple types of relations. These relations encompass connections within entities of the same type and connections between entities of different types. To represent such complicated networks, a sequence of graph snapshots $\mathbf{G} = \{\mathbf{G}^{(t)} \mid t \in \{1, 2, \cdots, T\}\}$ is proposed, where each graph snapshot $\mathbf{G}^{(t)} = \{\mathbf{V}, \mathbf{E}, \mathbf{F}\}$ contains node sets of different types $\mathbf{V} = \{\mathbf{V_n} \mid n \in \{1, 2, \cdots, N\}\}$, edge sets $\mathbf{E} = \{\mathbf{E_{jk}} \mid j, k \in \{1, 2, \cdots, N\}\}$ between two nodes in $\mathbf{V_j}$ and $\mathbf{V_k}$, and node attribute matrices $\mathbf{F} = \{\mathbf{F_n} \mid n \in \{1, 2, \cdots, N\}\}$ of each node set. Specifically, for edge set $\mathbf{E_{jk}}$, if $j \neq k$, $\mathbf{E_{jk}}$ present the bipartite relations of node pairs between $\mathbf{V_j}$ and $\mathbf{V_k}$. If $j = k$, $\mathbf{E_{jk}}$ present the intra-relations within node set $\mathbf{V_j}/\mathbf{V_k}$.

The research problem is set up as demonstrated in Fig. 2. The task is to predict the popularity trend on entities of a certain target type m, quantified by a label $\mathbf{L}_m$. The proposed method first learns the embeddings $\mathbf{Z}_m^{(t)}$ of nodes in $\mathbf{V}_m$ of each graph snapshot $\mathbf{G^{(t)}}$ by an encoder $\mathcal{G}$. Specifically, node embedding $\mathbf{Z}_m^{(t)}$ is obtained based on the node attribute matrices $\mathbf{F}$ and edges $\{\mathbf{E_{mj}} \mid j \in \{1, 2, \cdots, N\}\}$ connecting the target nodes, as formulated in (1):

$$\mathcal{G} : (\mathbf{V}, \{\mathbf{E_{mj}} \mid j \in \{1, 2, \cdots, N\}\}, \mathbf{F}) \rightarrow \mathbf{Z}_m^{(t)}. \tag{1}$$

Labels $\mathbf{l}_m^{(t)} \in \mathbf{L}_m$ are assigned to nodes in $\mathbf{V}_m$ of each graph snapshot $\mathbf{G^{(t)}}$. The label indicates the target value to be predicted. We aim to using learned node embeddings $\mathbf{Z}_m^{(t)}$ and known node labels $\mathbf{l}_m^{(t)}$ to predict the node labels $\hat{\mathbf{l}}_m^{(t+1)}$ in the upcoming graph snapshot $\mathbf{G^{(t+1)}}$ by a decoder $\mathcal{F}$, as formulated in (2):

$$\mathcal{F} : (\mathbf{Z}_m^{(t)}, \mathbf{l}_m^{(t)}) \rightarrow \hat{\mathbf{l}}_m^{(t+1)}. \tag{2}$$

Methodology

Previous studies regarding popularity trend prediction tasks in social networks suffer from neglecting the latent influence of complex relationships among entities. To overcome the limitations in prior work, this work introduces an advanced multi-layer temporal GNN framework designed to effectively capture the intricate relations and learn the representation embeddings of target entities in social networks.

In the following, we first demonstrate the overall framework of the proposed method. Then, we explain the details of the key components. Finally, we analyze the computational complexity of the proposed methods.

Overview

Figure 3 illustrates an overview of the proposed method. The input is a timed sequence of graph snapshots $\mathbf{G} = \{\mathbf{G^{(t)}} \mid t \in \{1, 2, \cdots, T\}\}$. Each graph snapshot contains adjacency matrices $\mathbf{A}$, which is a tensor for expressing multiple adjacency matrices, along with a node attribute matrix $\mathbf{F}$. The input adjacency matrices encompass 'intra graphs' $\mathbf{A_{in}}$ and

'bipartite graphs' $\mathbf{A_{bi}}$. Intra graphs are homogeneous, signifying the multiple relations within the target entities of the same type, while bipartite graphs represent the relationship between entities of different types. Node attributes $\mathbf{F}$ are shared across all the graphs, with only edge weights varying between individual graphs.

This framework mainly has four key components: graph structure learning (gsl) module, node aggregation module, time-sequence unit, and multi-head attention layer. First, input graphs $\mathbf{A}$ are enhanced by the GSL module, which is employed to improve the quality of input adjacency matrices by a refined graph structure $\hat{\mathbf{A}}$ learned from the node embeddings $\mathbf{Z}^{(t-1)}$ of the last iteration. Next, the enhanced adjacency matrices $\mathbf{A^*_{in}}$ and $\mathbf{A^*_{bi}}$ are fed to Node Aggregation module to learn the hidden node embeddings $\mathbf{H}^{(t)}_{in}$ and $\mathbf{H}^{(t)}_{bi}$ by the information on neighbor nodes. Then, $\mathbf{H}^{(t)}_{in}$ and $\mathbf{H}^{(t)}_{bi}$ are sent to the time-sequence unit to incorporate the temporal information $\mathbf{H}^{*(t-1)}_{in}$ between the adjacent graph snapshots. Finally, a multi-head attention layer receives the merged hidden node embeddings $\mathbf{H}^{*(t)}_{in}$ and node attributes $\mathbf{F}$ for the learning of node embeddings. The learned node embeddings $\mathbf{Z}^{(t)}$ are responsible for predicting the node labels $\hat{\mathbf{l}}^{(t+1)}$, representing the target attribute for popularity trend prediction.

The details of key components are introduced in the following:

Graph structure learning

Noisy or incomplete graphs can often lead to suboptimal representations, hindering the efficient learning of node embeddings, especially when dealing with real-world graph data. GSL is designed to jointly learn an optimized graph structure and corresponding adjacency matrix [36]. Based on the assumption that edges tend to connect nodes with similar representation, the proposed method refines the input graphs by computing the similarity between the last updated node embedding pairs, which are then used as the edge weights.

Euclidean distance is no longer an effective metric for graph node similarity. Instead, a generalized adaptive Mahalanobis distance is employed to quantify node similarity, as formulated in (3):

$$\phi(\mathbf{z}_i, \mathbf{z}_j) = \sqrt{(\mathbf{z}_i - \mathbf{z}_j)^T \mathbf{W}_d \mathbf{W}_d^T (\mathbf{z}_i - \mathbf{z}_j)}, \tag{3}$$

where $\mathbf{z}_i, \mathbf{z}_j$ denote the embedding of two node i and j learned in the last graph snapshot, and $\mathbf{W}_d$ is a trainable weight. $\mathbf{W}_d \mathbf{W}_d^T$ constructs a symmetric positive semi-definite trainable parameter. Mahalanobis distance can adaptively fit the task and the node embeddings during training. Then, this distance is exploited to calculate the refined adjacency matrix by exploiting the Gaussian kernel:

$$\hat{\mathbf{A}}_{\mathbf{ij}} = \exp\left(\frac{-\phi(\mathbf{z}_i, \mathbf{z}_j)}{2\sigma^2}\right), \tag{4}$$

where $\hat{\mathbf{A}}_{\mathbf{ij}}$ denote the elements in the refined adjacency matrix $\hat{\mathbf{A}}$, and σ denote the size of Gaussian kernel.

After obtaining the refined graph adjacency matrix $\hat{\mathbf{A}}$, the residual connections [37] is used to incorporate the original input adjacency matrices $\mathbf{A}$ by a hyperparameter α, as formulated in (5):

$$\mathbf{A}^* = \alpha\hat{\mathbf{A}} + (1-\alpha)\mathbf{A}, \tag{5}$$

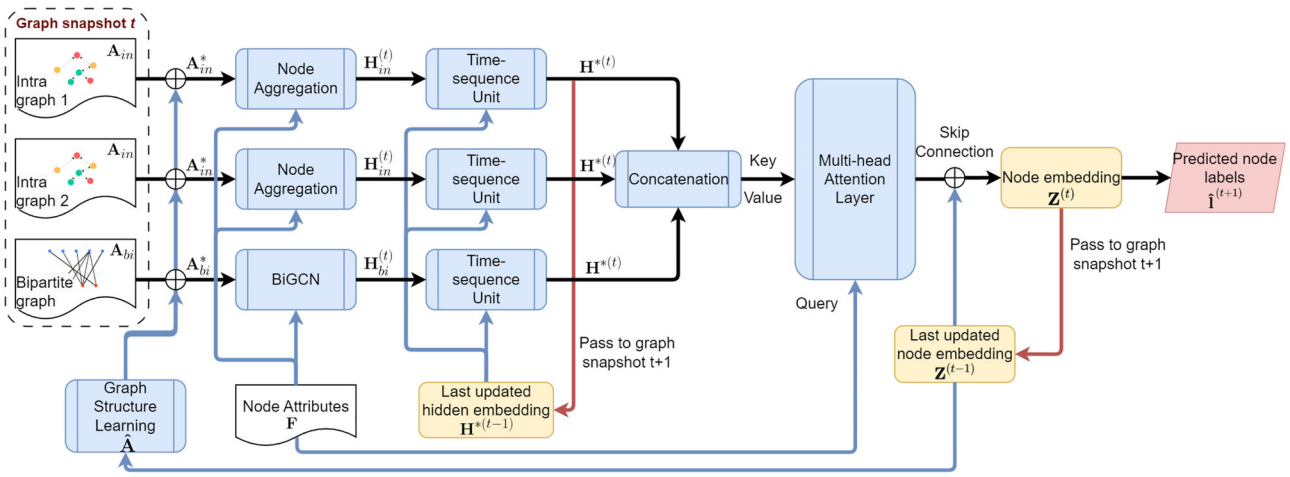


Fig. 3 Overview of the proposed multi-layer temporal GNN framework. Inputs are depicted as white blocks, main components are highlighted in blue, intermediate results are presented in yellow, and final outputs are shown in red. Intra graphs comprise the multiple relations within the target entities of the same type. The bipartite graph indicates the relationship between entities of different types

where α is used to mediate the influence of each part, and $\mathbf{A}^*$ is the finally improved adjacency matrix.

Node aggregation

The core of GNN is recurrently aggregating information from neighbor nodes. The proposed method uses the graph convolutional layer [38] to aggregate the neighbor node information in intra graphs $\mathbf{A}^*_{\mathbf{in}}$, as formulated in (6):

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}^*_{\mathbf{in}}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l+1)}_{in}\right), \tag{6}$$

where $\tilde{\mathbf{A}^*_{\mathbf{in}}} = \mathbf{A}^*_{\mathbf{in}} + \mathbf{I}$, $\tilde{\mathbf{D}}$ is the degree diagonal matrix of $\tilde{\mathbf{A}_{\mathbf{in}}}*$, $\mathbf{H}^{(l)}$ is the node embedding matrix in lth layer, $\mathbf{H}^{(0)}$ is initialized with the node feature matrix $\mathbf{F}$, $\mathbf{W}^{(l)}_{in}$ is the trainable weight matrix in lth layer, and $\sigma(\cdot)$ is the activation function. The result of the last graph convolutional layer is stored as the output of node aggregation on intra graphs, which is denoted as $\mathbf{H}^{(t)}_{in}$. t denotes the tth graph snapshot $\mathbf{G}^{(t)}$.

Specifically, for bipartite graph $\mathbf{A}^*_{\mathbf{bi}}$ that models the relation between the target node set $\mathbf{V}_m$ and other types of node set $\mathbf{V}_k$, a dedicated bipartite GCN layer [39] is exploited to separately execute graph convolutional operations on $\mathbf{V}_m$ and $\mathbf{V}_k$, as formulated in (7) and (8):

$$\mathbf{H}^{(l+1)}_m = \sigma\left(\left[\mathbf{D}^{-1}_m\mathbf{B}_m\mathbf{H}^{(l)}_k\mathbf{W}^{(l+1)}_m \parallel \mathbf{F}_m\boldsymbol{\omega}^{(l+1)}_j\right]\right) \tag{7}$$

$$\mathbf{H}^{(l+1)}_k = \sigma\left(\left[\mathbf{D}^{-1}_k\mathbf{B}_k\mathbf{H}^{(l)}_m\mathbf{W}^{(l+1)}_k \parallel \mathbf{F}_k\boldsymbol{\omega}^{(l+1)}_k\right]\right), \tag{8}$$

where $\mathbf{B}_m \in \mathcal{R}^{\|\mathbf{V}_m\|\times\|\mathbf{V}_k\|}$ and $\mathbf{B}_k \in \mathcal{R}^{\|\mathbf{V}_k\|\times\|\mathbf{V}_m\|}$ are the incidence matrix for two node sets $\mathbf{V}_m$ and $\mathbf{V}_k$ in the bipartite graph $\mathbf{A}^*_{\mathbf{bi}}$, respectively, $\mathbf{D}_m = \mathrm{Diag}(\sum_i \mathbf{B}_{m(1,i)}, \ldots, \sum_i \mathbf{B}_{m(\|\mathbf{V}_m\|,i)})$ and $\mathbf{D}_k = \mathrm{Diag}(\sum_i \mathbf{B}_{k(1,i)}, \ldots, \sum_i \mathbf{B}_{k(\|\mathbf{V}_k\|,i)})$ are the diagonal degree matrices of $\mathbf{V}_m$ and $\mathbf{V}_k$, $\mathbf{H}^{(l)}_m$ and $\mathbf{H}^{(l)}_k$ are the learned node embedding matrix in the lth layer. $\mathbf{W}^{(l)}_m$, $\mathbf{W}^{(l)}_k$, $\boldsymbol{\omega}^{(l)}_m$, and $\boldsymbol{\omega}^{(l)}_k$ are trainable parameters in the lth layer. $\parallel$ is the concatenation operation.

The BiGCN layer discards the result $\mathbf{H}^{(l)}_k$ of other node set, and only outputs the result $\mathbf{H}^{(l)}_m$ on the target node set $\mathbf{V}_m$. The result is denoted as $\mathbf{H}^{(t)}_{bi}$, where t denotes the tth graph snapshot $\mathbf{G}^{(t)}$.

Time-sequence unit

Time-sequence Unit receives the most recently updated node embeddings from the previous graph snapshot and transmits these learned node embeddings to the subsequent graph snapshot. An LSTM cell [40] is implemented to achieve this function:

$$\mathbf{h}_t, C_t = \mathrm{LSTMCell}(\mathbf{X}_t, \mathbf{h}_{t-1}, C_{t-1}) \tag{9}$$

$$\mathbf{H}^{*(t)}, C_t = \mathrm{LSTMCell}(\mathbf{H}^{(t)}, \mathbf{H}^{*(t-1)}, C_{t-1}). \tag{10}$$

LSTM cell has three inputs and two outputs: input embedding $\mathbf{X}_t$, input hidden state $\mathbf{h}_{t-1}$, input cell state C_{t-1}, output hidden state $\mathbf{h}_t$, and output cell state C_t, as formulated in (9). Outputs from Node Aggregation module $\mathbf{H}^{(t)}_{in}$ and $\mathbf{H}^{(t)}_{bi}$ (denoted as $\mathbf{H}^{(t)}$ for convenience in the following) are fed into the LSTM cell as $\mathbf{X}_t$, and the updated hidden embeddings $\mathbf{H}^{*(t-1)}$ from last graph snapshot $\mathbf{G}^{(t-1)}$ are fed into LSTM cell as the input hidden state $\mathbf{h}_{t-1}$. The output hidden state $\mathbf{h}_t$ will be stored as the updated hidden embeddings $\mathbf{H}^{*(t)}$. $\mathbf{H}^{*(t)}$ and cell state C_t are passed to the following graph snapshot $\mathbf{G}^{(t+1)}$. The LSTM cells are implemented for each individual graph. The inputs and outputs are formulated in (10).

Multi-head attention layer

After the time-sequence Unit, a concatenation layer is exploited to merge the updated hidden embeddings from each LSTM cell as a single hidden embedding matrix, denoted as $\mathbf{H}^{*(t)}$. Then, a multi-head attention layer is exploited to obtain the node embeddings. Multi-head attention mechanism originated from the famous Transformer [41] and proved efficient in learning temporal node representation [42–44].

The multi-head attention layer plays an efficient role in aggregating neighbor node embeddings and node attributes. The layer operates by computing the dot product of a query vector with a set of key vectors, ultimately generating a weight vector that assigns importance scores to the corresponding value vectors. The mechanism is defined as follows:

$$\mathbf{Q} = \mathbf{F}\mathbf{W}_Q \tag{11}$$

$$\mathbf{K} = \mathbf{H}^{*(t)}\mathbf{W}_K, \mathbf{V} = \mathbf{H}^{*(t)}\mathbf{W}_V \tag{12}$$

$$\mathrm{head}_i = \mathrm{Attn}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i) \tag{13}$$

$$= \mathrm{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^T}{\sqrt{d}}\right)\mathbf{V}_i \tag{14}$$

$$\mathrm{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Concat}(\mathrm{head}_1, \mathrm{head}_2 \cdots, \mathrm{head}_h)\mathbf{W}. \tag{15}$$

In the multi-head attention layer, the node hidden embedding $\mathbf{H}^{*(t)}$ is passed to the key and value parameters $\mathbf{K}$ and $\mathbf{V}$, and the node attribute matrix $\mathbf{F}$ is passed to the query parameter $\mathbf{Q}$. $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ and $\mathbf{W}$ are trainable parameters. Attn$(\cdot)$ is a scaled dot-product attention encoder implemented by the softmax function, where d denotes the dimension of the query parameter. MultiHead$(\cdot)$ concatenates the attention score for each head into a single attention score matrix.

Besides, a skip-connection is exploited to add the output attention score matrix MultiHead$(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ and the last updated node embedding matrix $\mathbf{Z}^{(t-1)}$ together, which is passed from the last graph snapshot $\mathbf{G}^{(t-1)}$. The skip-

connection is formulated in (16):

$$\mathbf{Z}^{(t)} = \mathbf{Z}^{(t-1)} + \text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \tag{16}$$

$$= \mathbf{Z}^{(t-1)} + \text{MultiHead}(\mathbf{F}, \mathbf{H}^{*(t)}, \mathbf{H}^{*(t)}) \tag{17}$$

The result of skip-connection $\mathbf{Z}^{(t)}$ is the final obtained node embedding matrix.

Loss function

The obtained node embedding $\mathbf{Z}^{(t)}$ is used to predict the node labels $\hat{\mathbf{l}}^{(t+1)}$ by exploiting an two-layer MLP decoder. The predicted node labels $\hat{\mathbf{l}}^{(t+1)}$ need to be as close as the target value of popularity trend prediction $\mathbf{l}^{(t+1)}$ in the next graph snapshot $\mathbf{G}^{(t+1)}$. Thus, it is a regression problem between the predicted value $\hat{\mathbf{l}}^{(t+1)}$ and actual value $\mathbf{l}^{(t+1)}$. The Mean Squared Error(MSE) loss is exploited to train the parameters:

$$\mathcal{L}_{\text{reg}} = \text{MSELoss}\left(\hat{\mathbf{l}}^{(t+1)}, \mathbf{l}^{(t+1)}\right) \tag{18}$$

$$= \frac{1}{N}\sum_{i=1}^{N}\left(\hat{\mathbf{l}}_i^{(t+1)} - \mathbf{l}_i^{(t+1)}\right), \tag{19}$$

where N denotes the total number of nodes in the training set.

Besides, the learnable parameter in the GSL module also needs restraints to accelerate the training and increase the stability of the learned graph topology structure. An MSE loss is exploited to restrain the gap between the refined $\hat{\mathbf{A}}$ and the original $\mathbf{A}$ for each graph:

$$\mathcal{L}_{gsl} = \sum^{\|\mathbf{A}\|}\text{MSELoss}(\hat{\mathbf{A}}, \mathbf{A}) \tag{20}$$

$$= \sum^{\|\mathbf{A}\|}\frac{1}{N}\sum_{i=1}^{N}(\hat{\mathbf{A}}_i - \mathbf{A}_i), \tag{21}$$

where $\|\mathbf{A}\|$ denotes the number of adjacency matrices in the tensor $\mathbf{A}$.

Finally, the total loss is

$$\mathcal{L} = \mathcal{L}_{\text{reg}} + \lambda\mathcal{L}_{\text{gsl}}, \tag{22}$$

where λ is a hyperparameter for balancing two losses.

Computational complexity

This section conducts a computational complexity analysis of the proposed method, with a particular focus on key components, including the GSL module, node aggregation module, time-sequence unit, and the multi-head attention layer. Regarding Eqs. (3), (4), (6)–(15), the complexity is primarily dominated by matrix multiplication. For convenience, this analysis assumes that the average node number of each node set is N, and both the node attributes $\mathbf{F}$ and node embeddings ($\mathbf{H}^{(t)}$ and $\mathbf{Z}^{(t)}$) share the same hidden dimension D. The computational complexity is calculated as follows:

The complexity of the GSL module is $\mathcal{O}(N^2D^2)$. The complexity of the graph convolutional layer in Node Aggregation is $\mathcal{O}(N^2D + ND^2)$. The complexity of the LSTM cell is $\mathcal{O}(ND)$. The complexity of the multi-head attention layer is $\mathcal{O}(N^2D + ND^2)$. GSL module costs the most time complexity, followed by the node aggregation module and multi-head attention layer. Time-sequence unit costs the least time complexity. As a result, the total complexity of the proposed method is $\mathcal{O}(N^2D^2) + \mathcal{O}(N^2D + ND^2) + \mathcal{O}(ND) + \mathcal{O}(ND^2) + \mathcal{O}(N^2D + ND^2) = \mathcal{O}(ND(ND + 2N + 3D + 1)) = \mathcal{O}(N^2D^2)$.

Table 1 Statistics of the social media network data sets used in our experiments

Data set	YouTube	MOOC	Reddit	Wikipedia
Durations	16 months	1 month	1 month	1 month
# Items	1358	98	1000	1000
# Users	13.6 k	7047	10,000	8227
# Edges	49.9 k	411,749	672,447	157,474

Experiments

In this section, the effectiveness of the proposed method is validated in real-world popularity trend prediction tasks. Experiments are conducted on four social media network data sets and evaluate the performance of the proposed method against baseline methods. Additionally, the sensitivity of the model hyperparameters is evaluated. Furthermore, an ablation study is performed to assess the significance of the key modules within the proposed method.

Data set description

The real-world social media network data sets used in the experiments are listed in Table 1.

- **YouTube live streaming data set**[1] [45]: this public data set comprises information on 1,358 live streaming channels and more than 136,000 viewers, spanning from April 2021 to July 2022. The attributes of both channels and viewers are represented as vectors. Relations within channels include the rate of common viewers, the conflict in streaming time, and the similarity of streaming

[1] https://www.kaggle.com/datasets/uetchy/vtuber-livechat.

content. Relations between channels and viewers present the interaction of donations and the amount of income. The data set is separated into several graph snapshots, each separated by a 1-month interval. The objective of the popularity trend prediction task within this data set is to forecast the donation income for each channel in the upcoming months.

- **MOOC students and courses**[2] [46]: this public data set encompasses interactions performed by students on MOOC (Massive Open Online Course) platform, involving 7047 users engaging with 98 courses, resulting in a total of 411,749 interactions. Both student and course attributes are represented as feature vectors. The data set is separated into several graph snapshots, each separated by a 2-day interval. The objective of the popularity trend prediction task within this data set is to predict the frequency of visits to each course in the forthcoming days.
- **Reddit post data set**[3] [46]: this public data set comprises one month of posts from 10,000 active users on the 1000 most active topics on the Reddit community forum, resulting in 672,447 interactions. The text of each post is converted into feature vectors. The data set is separated into several graph snapshots, each separated by a 2-day interval. The objective of the popularity trend prediction task within this data set is to predict the post numbers on each topic in the upcoming days.
- **Wikipedia edits**[4] [46]: this public data set encompasses one month of edits made to Wikipedia pages, comprising the 1000 most frequently edited pages. It involves 8227 editors and a total of 157,474 edit records. Like the Reddit data set, the edit text is transformed into feature vectors. The data set is separated into several graph snapshots, each separated by a 2-day interval. The objective of the popularity trend prediction task within this data set is to predict the frequency of edits on each page in the forthcoming days.

This work aims to predict values that reflect the popularity degree of target entities. These values correspond to the amount of income (YouTube live streaming data set), the number of visits (MOOC and Reddit data sets), and the number of edits (Wikipedia data set). However, these values often vary across different magnitudes, ranging from 1 to more than 10 million in some cases. For example, the distribution of target values in the YouTube live streaming data set is illustrated in Fig. 4. Predicting target values across such a large and imbalanced distribution of magnitudes poses a challenge for the proposed model, potentially resulting in significant biases in the prediction results.

To address this challenge, we implement a transformation on the prediction target values, converting them into percentages of changes. Thus, the proposed model predicts the percentage of change in the popularity degree values rather than the original values. As depicted in Fig 5, the percentages only range from -1 to 2 after eliminating outliers. Consequently, the proposed model can provide more accurate predictions within this smaller range. This approach allows us to mitigate the negative impact stemming from the vast magnitudes of target values, which are influenced by confounding effects.

Setup of the experiment

The data sets introduced above provide the intra graphs of item entities, bipartite graphs representing user-item relationships, and feature vectors for the input. Specifically, in cases where the data sets solely comprise bipartite graphs of user-item pairs, the intra graph is constructed based on the two-hop neighborhood of item entities. It means that item entities are interconnected in the intra graph if they share connections with the same user entity in the bipartite graph.

Experiments are executed on a platform with Intel Xeon Platinum 8360Y CPU and NVIDIA A100 for NVLink 40GiB HBM2 GPU. The standard model hyperparameters are fine-tuned, including the interval of graph snapshots, learning rate, epochs, and dropout rate. Regarding the data set configuration, we test different choices of the interval of graph snapshots for each data set and finally determine the choice with the best performance: a 1-month interval for the YouTube live streaming data set and a 2-day interval for the MOOC, Reddit, and Wikipedia data sets. During training, the Adam optimizer is employed with a learning rate set to 0.01. The model is trained for 30 epochs, including training, validation, and testing phases. A dropout rate of 0.2 is applied. The multi-head attention layer configures two attention heads. Furthermore, an early stopping strategy is implemented, which halts training if the validation loss fails to decrease for five consecutive iterations. Additionally, sensitivity experiments are performed for specific hyperparameters through grid search, as explained later.

All the configurations above are determined by manual grid search. Regarding the sensitivity analyses of the configurations above, the interval of graph snapshots severely impacts the model performances because the information in graph snapshots is totally different with distinct intervals. In comparison, the learning rate and the dropout rate are less sensitive to the experimental results. Finally, as introduced above, the number of attention heads in the multi-head attention layer significantly impacts the computational costs.

[2] https://snap.stanford.edu/jodie/.

[3] https://snap.stanford.edu/jodie/.

[4] https://snap.stanford.edu/jodie/.

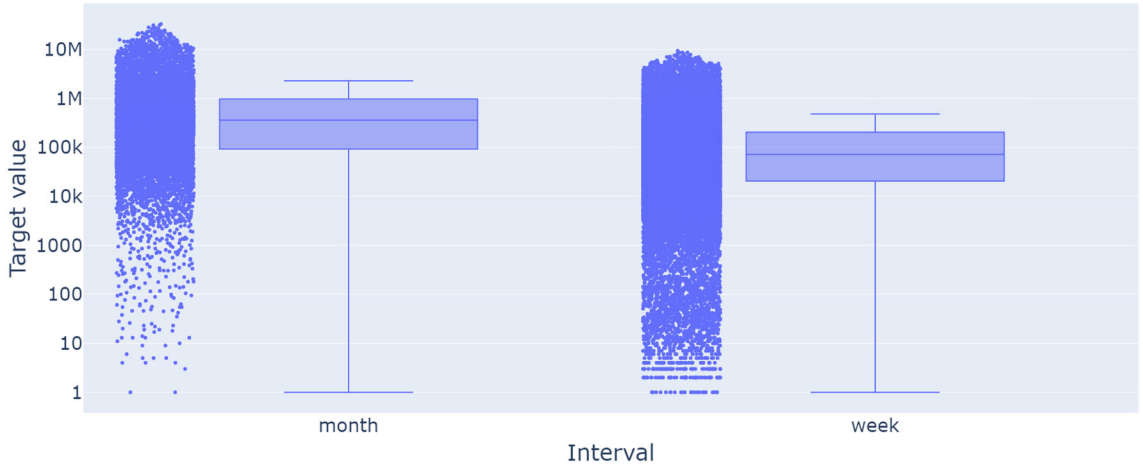


Fig. 4 The box plot of the target values (true labels) in the YouTube live streaming data set. The target values are distributed from 1 to more than 10 million. "Month" and "Week" denotes the interval of graph snapshots (1-month interval and 1-week interval)

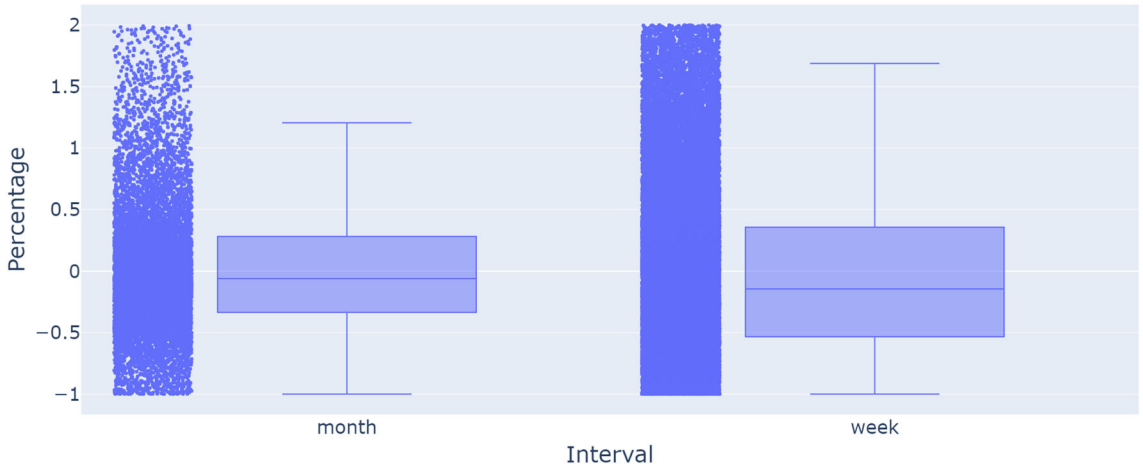


Fig. 5 The box plot of the improved percentage of change on target values (true labels) in YouTube live streaming data set. The percentages are distributed from −1 to 2 after eliminating outliers. "Month" and "Week" denotes the interval of graph snapshots (1-month interval and 1-week interval)

Table 2 Input information required by the baseline models

	Node attributes	Graph structure	Temporal information
Linear regression algorithm			
GBR	✓	×	×
XGBoost	✓	×	×
MLP	✓	×	×
Time-sequence model			
LSTM-FCN	✓	×	✓
Static heterogeneous graph method			
HetGNN	✓	✓	×
HAN	✓	✓	×
Temporal heterogeneous graph methods			
HGT	✓	✓	✓
CoupledGNN	✓	✓	✓
Proposed method	✓	✓	✓

Baselines

The following baseline methods are included for comparison in experiments, including basic linear regression approaches, time-sequence models, previous popularity trend prediction methods, and recently proposed heterogeneous GNN methods:

1. **Gradient Boost Regressor (GBR)**: A gradient boost regressor from the scikit-learn toolkit.
2. **XGBoost**: An ensemble gradient boosting decision tree model from XGBoost library.
3. **Multi-layer Perceptron (MLP)**: A basic two-layer fully connection neural network.
4. **LSTM-FCN**[5] [47]: A time-sequence deep learning model combining LSTM and a fully convolutional network.
5. **HetGNN**[6] [18]: A GNN model for learning node embedding representations in heterogeneous graphs.
6. **HAN**[7] [17]: A heterogeneous graph attention network based on the hierarchical node-level and semantic-level attentions.
7. **HGT**[8] [48]: A heterogeneous graph transformer architecture that can deal with large-scale heterogeneous and temporal graphs.
8. **CoupledGNN**[9] [7]: A model solves the network-aware popularity prediction problem, capturing the cascading effect explicitly by two coupled GNNs.

Table 2 presents the required input information for each baseline method. The default parameter settings are maintained during training and testing for all baseline methods. The specific experimental configurations are as follows:

- Linear regression algorithms (GBR and XGBoost) and the Neural Network (MLP) receive every feature vector and learn to predict the target value.
- Time-sequence models (LSTM–FCN) handle the time-sequence data, taking in sequences of feature vectors as input. These models generate a sequence of hidden state embeddings, which are subsequently used for predicting the target value.
- Static heterogeneous graph methods (HetGNN and HAN) are typically applied to model graph-structured data containing multiple types of entities and relations. In the case of this work, the user-item relations in the data sets can be viewed as a unique heterogeneous graph. These models receive feature vectors and the graph structure information as input.
- Temporal heterogeneous graph methods (HGT and CoupledGNN) operate on dynamic heterogeneous graphs encompassing node attributes, graph structure, and temporal information. These models are considered the strongest baseline methods compared to the proposed method.

[5] https://github.com/titu1994/LSTM-FCN.

[6] https://github.com/chuxuzhang/KDD2019_HetGNN.

[7] https://github.com/Jhy1993/HAN.

[8] https://github.com/UCLA-DM/pyHGT.

[9] https://github.com/CaoQi92/CoupledGNN.

Evaluation

Table 3 and Fig. 6 present the experimental results for the popularity trend prediction tasks. The performance evaluation is based on the Rooted Mean Squared Error (RMSE) score and the Mean Absolute Percentage Error (MAPE):

$$\mathrm{RMSE}(y, \hat{y}) = \sqrt{\frac{\sum_{i=i}^{N} (y_i - \hat{y}_i)^2}{N}} \tag{23}$$

$$\mathrm{MAPE}(y, \hat{y}) = \frac{1}{N} \sum_{i=i}^{N} \frac{\| y_i - \hat{y}_i \|}{\| y_i \|}, \tag{24}$$

which quantify the average difference between predicted values $\hat{y}$ and actual values y of total N samples. Smaller RMSE and MAPE indicate superior performance. The proposed method has the best overall performance, achieving the lowest RMSE and MAPE scores on all four data sets, as indicated by bold font in the table. The respective performance of the baselines is evaluated as follows:

- Linear regression algorithms (GBR and XGBoost) and the Neural Network (MLP) rely solely on the attribute vectors of items, which is often impractical for predicting trends.
- Time-sequence models (LSTM–FCN) take in sequences of attribute vectors and generate corresponding sequences of hidden state embeddings for each graph snapshot. However, these models are susceptible to input noise in the early steps, which can significantly impact subsequent outputs. Consequently, relying solely on long sequences of attribute vectors may not perform as well. In some cases, they may even underperform compared to linear regression algorithms that only use attribute vectors.
- Static heterogeneous GNN methods (HetGNN and HAN) are good at analyzing graph structural information and edge interactions among different types of nodes. These methods outperform linear regression methods and time-sequence models. However, they are not designed to handle temporal data, a crucial aspect of popularity trend prediction.
- Temporal heterogeneous GNN methods (HGT and CoupledGNN) receive inputs similar to the proposed method.

Table 3 The results of node attribute value prediction tasks on four data sets

Model	YouTube		MOOC		Reddit		Wikipedia	
Metrics	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
Linear regression algorithm								
GBR	0.524	59.747	0.095	2.137	0.122	3.839	0.040	2.412
XGBoost	0.535	61.814	0.095	2.012	0.123	3.578	0.042	2.684
MLP	0.536	62.281	0.505	2.182	0.152	4.569	0.143	2.192
Time-sequence model								
LSTM-FCN	0.557	103.931	0.153	2.002	0.106	4.288	0.158	4.128
Static heterogeneous graph method								
HetGNN	0.518	44.325	0.092	1.968	0.082	3.403	0.036	1.992
HAN	0.524	44.933	0.092	1.971	0.082	3.602	0.036	1.933
Temporal heterogeneous graph method								
HGT	0.512	40.526	0.091	1.976	0.079	3.246	0.032	2.029
CoupledGNN	0.510	40.193	0.090	1.971	0.071	3.110	0.030	1.992
Our method	**0.507**	**39.652**	**0.080**	**1.960**	**0.036**	**2.839**	**0.027**	**1.847**

Metric is RMSE scores and MAPE scores (%). Results are mean values of 30 runnings

Their performance depends on their underlying architectures and ability to effectively capture temporal and graph structure information. The proposed method outperforms HGT and CoupledGNN, owing to distinctive architectural choices such as incorporating GSL and multi-head attention layer.

Parameter sensitivity

In this section, an experiment is conducted to determine the optimal values for significant hyperparameters, namely the hyperparameter α in GSL, the balance hyperparameter λ in the loss function, and the dimension of hidden embeddings. These hyperparameters are fine-tuned through grid search within specific ranges.

In Fig. 7, the performance on four data sets is demonstrated concerning each hyperparameter, using the RMSE score as the metric. The rows represent the results for four data sets, while the columns correspond to values of α ranging from 0.01 to 0.9, λ ranging from 0.01 to 1.0, and hidden dimensions varying from 16 to 256.

Based on the results, the proposed method is not highly parameter-sensitive. The relatively optimal hyperparameter choices are $\alpha = 0.1$, $\lambda = 0.1$, and a hidden dimension of 128. These settings are employed in the experimental results for the popularity trend prediction tasks in the previous evaluation section.

Ablation study

An ablation study is conducted to assess the significance of key components in the proposed method, namely GSL, intra graphs, bipartite graphs, and the time-sequence unit. It is interesting to discover what component contributes the most to different application cases in experiments. Each of these components is responsible for specific types of input information:

- GSL enhances the quality of input graphs.
- Intra graphs represent intra-relations within target entities.
- Bipartite graphs depict relations between target entities and other types of entities.
- The time-sequence unit receives and propagates temporal information between adjacent graph snapshots.

In Table 4, the performance is demonstrated when each component is omitted, with the metric being the RMSE score. The results without specific components consistently underperform compared to the completed method, highlighting the indispensability of all components to the overall performance of the proposed method. The worst results in each data set are marked in bold font. These results indicate that the corresponding component is the most significant for various data sets. To provide specific insights:

- In the YouTube data set, the method without GSL exhibits the worst performance, highlighting the importance of improving input graph quality.
- In the MOOC data set, the absence of bipartite graph input leads to the poorest result, emphasizing the significance of item-user relations.
- In the Reddit data set, all results are equally achieved, suggesting that no single component stands out as the most crucial.

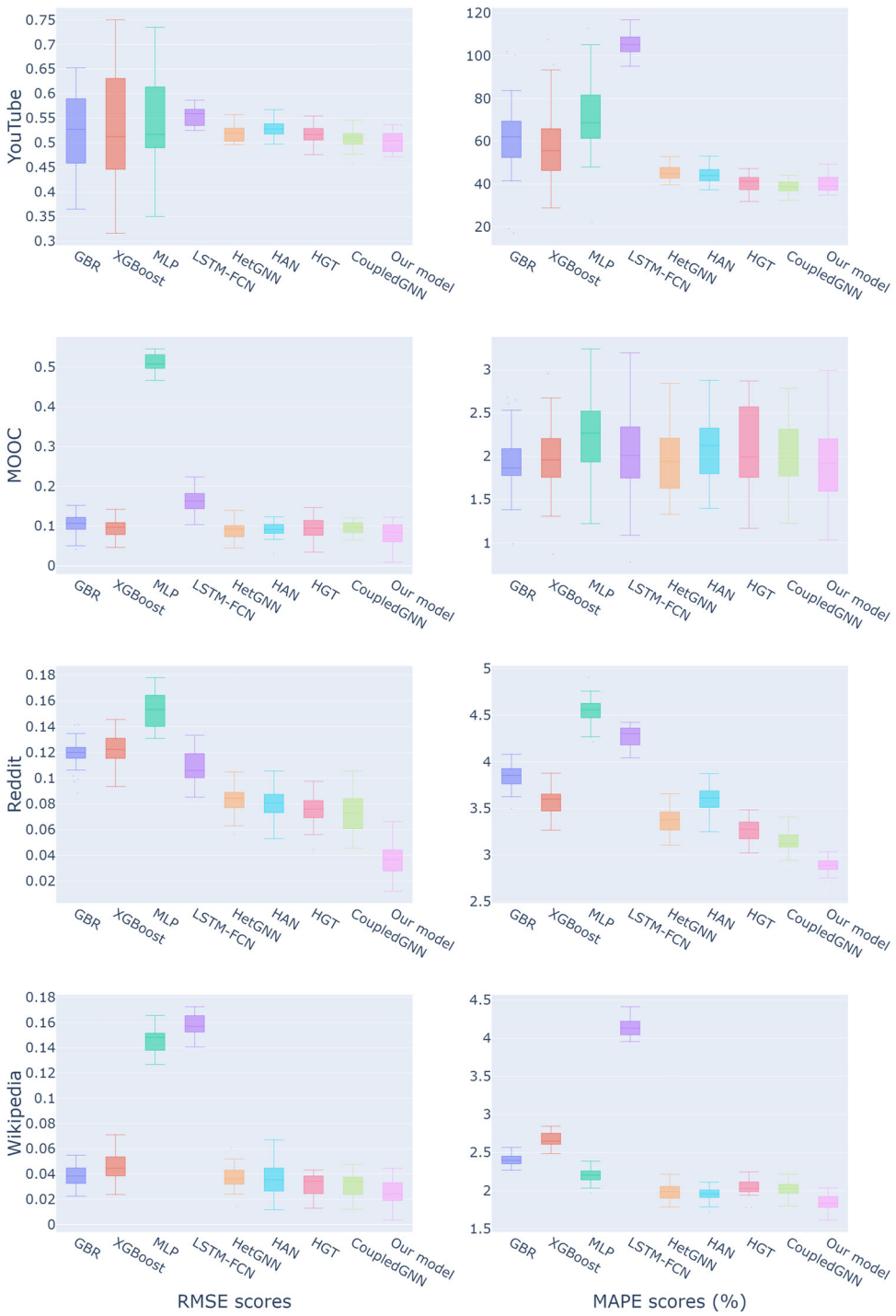


Fig. 6 The box plot of the results. Four rows denote four data sets and two columns denote two metrics

Table 4 The ablation study results without specific key components in the proposed method

	YouTube	MOOC	Reddit	Wikipedia
w/o bi-graph	0.537	**0.100**	0.039	0.029
w/o intra-graph	0.537	0.090	0.039	0.029
w/o time-sequence unit	0.536	0.086	0.039	**0.031**
w/o GSL	**0.538**	0.095	0.039	0.029
Completed method	0.507	0.080	0.036	0.027



Fig. 7 Study of parameter sensitivity

- In the Wikipedia data set, the most crucial component is the time-sequence unit, highlighting the valuable information in the edit history.

It is reasonable that the most significant components vary across different data sets, as the most abundant information differs in each case. In summary, those findings emphasize that all components are essential, and their significance depends on the specific application scenarios.

Statistical test

This section introduces a statistical test to determine the statistical significance of observed differences among baseline methods in "Evaluation" Section. The test involves formulating a null hypothesis H_0 and an alternative hypothesis H_1 as follows:

Hypothesis 0. *All the methods have the same performance.*
Hypothesis 1. *The performance of methods has significant differences.*

Next, we employ two statistical tests, the Friedman test and the Wilcoxon sign-rank test, to assess the differences between the proposed method and baseline methods and validate the formulated hypotheses.

Friedman test

The Friedman test, a nonparametric statistical method, is employed to identify significant differences in the performance of two or more methods across multiple test attempts [49, 50]. The Friedman test is the analog of the repeated measures Analysis of Variance (ANOVA) in nonparametric statistical procedures.

The initial step in calculating the Friedman test statistic involves converting the original results in Table 3 into ranks. Specifically, we assess the performance of nine methods (eight baseline methods and one proposed method) on four data sets (YouTube, MOOC, Reddit, and Wikipedia) using two metrics (RMSE score and MAPE score), resulting in a total of eight tests. The results are ranked for each test, ranging from 1 (indicating the best result) to 9 (representing the worst result). Subsequently, the average rank for each method is calculated based on all eight tests. Table 5 illustrates the ranks of the nine methods across the eight tests,

with tied ranks assigned an average value. By using these ranks, the Friedman statistic can be computed as follows:

$$\chi_f^2 = \frac{12n}{k(k+1)}\left[\sum_j R_j^2 - \frac{k(k+1)^2}{4}\right], \tag{25}$$

which follows a χ^2 distribution with $k-1$ degrees of freedom. Here, n represents the number of tasks, k is the count of baseline methods included in the comparison, and R_j denotes the average rank of each method j.

Iman and Davenport propose a derivation from the Friedman statistic [51] as follows:

$$F_{ID} = \frac{(n-1)\chi_f^2}{n(k-1)-\chi_f^2}, \tag{26}$$

which follows an F distribution with $k-1$ and $(k-1)(n-1)$ degrees of freedom.

Based on the formula above, the test statistic F_{ID} is calculated as 52.534, and the corresponding p value is 1.943×10^{-9}. As the p value is less than $\alpha = 0.05$, we reject the null hypothesis H_0, which indicates that all baseline methods exhibit the same performance. In other words, there is substantial evidence to support the presence of statistical significance among these methods.

Wilcoxon sign-rank test

While the Friedman test excels at detecting overall differences across multiple comparisons, its limitation lies in its inability to pinpoint significant differences within specific pairs of methods. To address this constraint, the Wilcoxon sign-rank test emerges as a non-parametric alternative to the paired t test, particularly suited for non-normally distributed samples in this work.

In the Wilcoxon sign-rank test, the differences between sample averages for all method pairs are calculated in multiple comparisons. The test is defined as follows:

$$R^+ = \sum_{d_i>0} \text{rank}(d_i) + \frac{1}{2}\sum_{d_i=0} \text{rank}(d_i) \tag{27}$$

$$R^- = \sum_{d_i<0} \text{rank}(d_i) + \frac{1}{2}\sum_{d_i=0} \text{rank}(d_i), \tag{28}$$

where d_i represents the difference between the performance scores of two methods on the ith out of n tests. R^+ is the sum of ranks for tests where the first algorithm outperformed the second, and R^- is the sum of ranks for the opposite. Ranks of $d_i = 0$, indicating ties, are evenly distributed between the sums. $T = \min(R^+, R^-)$ is the smaller of the sums. If T is less than the critical value from the Wilcoxon distribution for n degrees of freedom, the null hypothesis H_0 is rejected, signifying that the given method outperforms the other, with associated p values.

Table 5 The ranks of nine methods assessed on eight tests

	Proposed method	GBR	XGBoost	MLP	LSTM-FCN	HetGNN	HAN	HGT	Coupled GNN
T_1	1	5.5	7	8	9	4	5.5	3	2
T_2	1	6.5	6.5	9	8	4.5	4.5	3	2
T_3	1	7	8	9	6	4.5	4.5	3	2
T_4	1	6	7	8	9	4.5	4.5	3	2
T_5	1	6	7	8	9	4	5	3	2
T_6	1	8	7	9	6	2	4	5	3
T_7	1	7	5	9	8	4	6	3	2
T_8	1	7	8	6	9	3.5	2	5	3.5
Avg. rank	1	6.625	6.9375	8.25	8	3.875	4.5	3.5	2.3125

$T_1 - T_4$ represent RMSE scores on YouTube, MOOC, Reddit, and Wikipedia data sets. $T_5 - T_8$ represent MAPE scores on YouTube, MOOC, Reddit, and Wikipedia data sets. Any tied ranks are assigned an average rank

Table 6 The results of the Wilcoxon sign-rank test between pairs of the proposed method and each baseline method

Vs	GBR	XGBoost	MLP	LSTM-FCN	HetGNN	HAN	HGT	Coupled GNN
Adjusted p values	0.0122	0.0122	0.0122	0.0122	0.0122	0.0122	0.0122	0.0122

Results are Benjamini–Hochberg FDR adjusted p values

In this study, we employ the Python library *scikit-posthocs*[10] to conduct the Wilcoxon sign-rank test and calculate p values for each pairwise comparison between the proposed method and baseline methods. Similar to the Friedman test, this assessment involves ranking the performance of nine methods across eight tests.

To address the issue of inflated Type I error (familywise error rate) in multiple comparisons, the test incorporates adjusted p values using the Benjamini–Hochberg false discovery rate (FDR) method [52]. The results, including the calculated adjusted p values, are presented in Table 6.

All adjusted p values for the proposed method versus baseline methods are below the significance level of $\alpha = 0.05$, indicating statistical significance compared to these methods. It is noticeable that all the adjusted p values are all the same because the proposed method consistently achieves a rank of 1 across all eight tests, resulting in T always equaling 0.

Conclusions and discussion

Discussion

In this section, we discuss the improvements achieved by comparing our method to the most recent and influential studies concerning GNNs in real-world prediction tasks. In contrast to other state-of-the-art heterogeneous GNNs, such as HetGNN [18], HAN [17], HGT [48], and CoupledGNN [7], our method introduces a specialized multi-layer architecture. This architecture decomposes the intricate multiple relationship types into two categories: intra connections within entities of the same type and inter connections across entities of different types. This simplification process proves advantageous for handling heterogeneous graphs. Moreover, our method incorporates several modules to enhance overall performance, including the GSL module for improving the quality of input graphs and a multi-head attention layer for efficiently combining outputs from each graph layer.

In comparison to recent trend prediction studies unrelated to graphs, such as those in citation networks [8], terrorist attacks [34], and traffic flow [35], these methods often focus primarily on selecting characteristics and statistical features of target entities. They tend to overlook information diffusions along relationships and commonly rely on simplistic machine learning or statistical methods, resulting in suboptimal learning quality. To address these shortcomings, our method emphasizes the "message passing" along relationships within entities and employs advanced deep-learning-based encoders and decoders, leading to high learning efficiency.

[10] https://scikit-posthocs.readthedocs.io/en/latest/index.html.

In summary, our method addresses real-world trend prediction tasks and demonstrates remarkable improvements compared to the most recent and influential studies.

Conclusion

This work provides an effective solution for learning the representation of entities within social media networks and predicting trends in real-world scenarios. To achieve this, we develop an advanced multi-layer temporal GNN framework that captures information diffusion and temporal dynamics among different types of entities. The experimental results demonstrate the efficiency of the proposed method compared to various baseline methods, including basic linear regression approaches, time-sequence models, previous popularity trend prediction methods, and recently proposed heterogeneous GNN methods. Besides, massive experiments are conducted to explore the optimal choices of hyperparameters and assess the significance of key components in the proposed method.

In addition, this work makes a substantial contribution to the rapidly growing and promising field of trend prediction in social media. Social media is a primary source for obtaining information about emerging trends worldwide. By predicting trends in social media, the proposed method can assist users in gaining a deeper understanding of the future trajectories of public concerns and interests. Anticipating trends in advance holds significant value, as it enables individuals and organizations to stay ahead of the competition, align their studies and work in the right direction, and seize opportunities for informed decision-making in the future.

Future work

In terms of the limitations of this work, we acknowledge that the proposed method only supports input in the form of sequences of graph snapshots. The proposed method cannot handle other types of temporal graphs, such as sequences of graph actions accompanied by timestamps. Furthermore, real-world situations are highly dynamic and influenced by numerous factors. It is important to note that no one can accurately predict the future without any uncertainties or missing information. Our method is intended to provide results for reference purposes only. Any negative consequences resulting from the misuse of the method are not addressed in this work.

Acknowledgements This work is partly supported by Japan Science and Technology Agengcy Support for Pioneering Research Initiated by the Next Generation (JST SPRING) (Grant Number JPMJSP2106) and Japan Society for the Promotion of Science (JSPS) Grant-in-Aid for Scientific Research (Grant Numbers 23H03451, 21K12042). The part of this paper is based on results obtained from JPNP20017 commissioned

by New Energy and Industrial Technology Development Organization (NEDO).

Data availability The data set that support the findings of this study are available via following: YouTube live streaming: https://www.kaggle.com/datasets/uetchy/vtuber-livechat, MOOC, Reddit, and Wikipedia: https://snap.stanford.edu/jodie/.

Declarations

Conflict of interest The authors have no Conflict of interest to declare that are relevant to the content of this article.

Ethical approval This work adheres to the guidelines and ethical considerations outlined in the privacy policies of each social media platform involved in our study. We acknowledge the public nature and free accessibility of content posted on social network platforms, which are not password-protected and have thousands of active users. All analyses are conducted using publicly available data, and we do not attempt to track users across different platforms. In data preprocessing, we anonymized all user names or account IDs. Our data sets do not contain any information about individuals, and we have taken measures to ensure that our results do not disclose the identity of any specific account.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Wu L, Sun P, Fu Y, Hong R, Wang X, Wang M (2019) A neural influence diffusion model for social recommendation. In: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp 235–244
2. Chunaev P (2020) Community detection in node-attributed social networks: a survey. Comput Sci Rev 37:100286
3. Wang H, Gao Q, Li H, Wang H, Yan L, Liu G (2022) A structural evolution-based anomaly detection method for generalized evolving social networks. Comput J 65(5):1189–1199
4. Rousidis D, Koukaras P, Tjortjis C (2020) Social media prediction: a literature review. Multimed Tools Appl 79(9–10):6279–6311
5. Altshuler Y, Pan W, Pentland A (2012) Trends prediction using social diffusion models. In: Social Computing, Behavioral-Cultural Modeling and Prediction: 5th International Conference, SBP 2012, College Park, MD, USA, April 3-5, 2012. Proceedings 5. Springer, pp 97–104
6. Zeng A, Gualdi S, Medo M, Zhang Y-C (2013) Trend prediction in temporal bipartite networks: the case of movielens, netflix, and digg. Adv Complex Syst 16:1350024
7. Cao Q, Shen H, Gao J, Wei B, Cheng X (2020) Popularity prediction on social platforms with coupled graph neural networks. In: Proceedings of the 13th International Conference on Web Search and Data Mining. pp 70–78
8. Li C, Zhou F, Luo X, Trajcevski G (2021) Kernel-based structural-temporal cascade learning for popularity prediction. In: 2021 IEEE Global Communications Conference (GLOBECOM). pp 1–6
9. Cai H, Zheng VW, Chang KC-C (2018) A comprehensive survey of graph embedding: problems, techniques and applications. IEEE Trans Knowl Data Eng 30(9):1616–1637
10. Cui P, Wang X, Pei J, Zhu W (2018) A survey on network embedding. IEEE Trans Knowl Data Eng 31(5):833–852
11. Wu Z, Pan S, Chen F, Long G, Zhang C, Philip SY (2020) A comprehensive survey on graph neural networks. IEEE Trans Neural Netw Learn Syst 32(1):4–24
12. Liben-Nowell D, Kleinberg J (2007) The link-prediction problem for social networks. J Am Soc Inf Sci Technol 58(7):1019–1031
13. Sankar A, Wu Y, Gou L, Zhang W, Yang H (2020) Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In: WSDM2020, pp 519–527
14. Gao C, Zhu J, Zhang F, Wang Z, Li X (2022) A novel representation learning for dynamic graphs based on graph convolutional networks. IEEE Trans Cybern
15. Jin R, Liu X, Murata T (2023) Predicting potential real-time donations in youtube live streaming services via continuous-time dynamic graphs. Mach Learn 1–35
16. Dong Y, Chawla NV, Swami A (2017) Metapath2vec: Scalable representation learning for heterogeneous networks. In: KDD, pp 135–144
17. Wang X, Ji H, Shi C, Wang B, Ye Y, Cui P, Yu PS (2019) Heterogeneous graph attention network. In: WWW, pp 2022–2032
18. Zhang C, Song D, Huang C, Swami A, Chawla NV (2019) Heterogeneous graph neural network. In: KDD, pp 793–803
19. Chairatanakul N, Liu X, Murata T (2021) Pgra: projected graph relation-feature attention network for heterogeneous information network embedding. Inf Sci 570:769–794
20. Kipf TN, Welling, M (2016) Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907
21. Zhang M, Chen Y (2018) Link prediction based on graph neural networks. In: NeurIPS2018. NIPS'18, pp 5171–5181
22. Zhang M, Cui Z, Neumann M, Chen Y (2018) An end-to-end deep learning architecture for graph classification. In: AAAI2018
23. Hamilton W, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. Adv Neural Inf Process Syst 30
24. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y (2018) Graph attention networks. arXiv preprint arXiv:1710.10903v3
25. Kipf TN, Welling, M (2016) Variational graph auto-encoders. arXiv preprint arXiv:1611.07308
26. Zangari L, Interdonato R, Calió A, Tagarelli A (2021) Graph convolutional and attention models for entity classification in multilayer networks. Appl Netw Sci 6(1):1–36
27. Leblay J, Chekol MW, Liu X (2020) Towards temporal knowledge graph embeddings with arbitrary time precision. In: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, pp 685–694
28. Yang J, Leskovec J (2011) Patterns of temporal variation in online media. In: Proceedings of the Fourth ACM International Conference on Web Search and Data Mining, pp 177–186
29. Zhao Q, Erdogdu MA, He HY, Rajaraman A, Leskovec J (2015) Seismic: A self-exciting point process model for predicting tweet popularity. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 1513–1522
30. Malakar S, Ghosh M, Bhowmik S, Sarkar R, Nasipuri M (2020) A ga based hierarchical feature selection approach for handwritten word recognition. Neural Comput Appl 32:2533–2552
31. Bacanin N, Zivkovic M, Al-Turjman F, Venkatachalam K, Trojovskỳ P, Strumberger I, Bezdan T (2022) Hybridized sine cosine

algorithm with convolutional neural networks dropout regularization application. Sci Rep 12(1):6302
32. Bacanin N, Stoean R, Zivkovic M, Petrovic A, Rashid TA, Bezdan T (2021) Performance of a novel chaotic firefly algorithm with enhanced exploration for tackling global optimization problems: Application for dropout regularization. Mathematics 9(21):2705
33. He X, Gao M, Kan M-Y, Liu Y, Sugiyama K (2014) Predicting the popularity of web 2.0 items based on user comments. In: Proceedings of the 37th International ACM SIGIR Conference on Research & Development in Information Retrieval, pp 233–242
34. Hou Z, Zhou Y, Wu X, Bu F (2023) Attention-based spatial-temporal multi-graph convolutional networks for casualty prediction of terrorist attacks. Complex Intell Syst 9:1–22
35. Yang H, Li Z, Qi Y (2023) Predicting traffic propagation flow in urban road network with multi-graph convolutional network. Complex Intell Syst 10:1–13
36. Zhu Y, Xu W, Zhang J, Du Y, Zhang J, Liu Q, Yang C, Wu S (2021) A survey on graph structure learning: progress and opportunities. arXiv preprint arXiv:2103.03036
37. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp 770–778
38. Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907v4
39. Jin R, Xia T, Liu X, Murata T, Kim K-S (2021) Predicting emergency medical service demand with bipartite graph convolutional networks. IEEE Access 9:9903–9915
40. Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9:1735–80
41. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Advances in Neural Information Processing Systems, pp 5998–6008
42. Xu D, Ruan C, Korpeoglu E, Kumar S, Achan K (2020) Inductive representation learning on temporal graphs. arXiv preprint arXiv:2002.07962
43. Rossi E, Chamberlain B, Frasca F, Eynard D, Monti F, Bronstein M (2020) Temporal graph networks for deep learning on dynamic graphs. arXiv preprint arXiv:2006.10637
44. Wang X, Lyu D, Li M, Xia Y, Yang Q, Wang X, Wang X, Cui P, Yang Y, Sun B, et al (2021) Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In: SIGMOD2021, pp 2628–2638
45. Uechi Y (2022) VTuber 1B: large-scale live chat and moderation events dataset. https://holodata.org/vtuber-1b
46. Kumar S, Zhang X, Leskovec J (2019) Predicting dynamic embedding trajectory in temporal interaction networks. In: KDD2019, pp 1269–1278
47. Karim F, Majumdar S, Darabi H, Chen S (2018) Lstm fully convolutional networks for time series classification. IEEE Access 6:1662–1669
48. Hu Z, Dong Y, Wang K, Sun Y (2020) Heterogeneous graph transformer. In: Proceedings of the Web Conference 2020, pp 2704–2710
49. Friedman M (1937) The use of ranks to avoid the assumption of normality implicit in the analysis of variance. J Am Stat Assoc 32(200):675–701
50. Friedman M (1940) A comparison of alternative tests of significance for the problem of m rankings. Ann Math Stat 11(1):86–92
51. Iman RL, Davenport JM (1980) Approximations of the critical region of the Fbietkan statistic. Commun Stat Theory Methods 9(6):571–595
52. Benjamini Y, Hochberg Y (1995) Controlling the false discovery rate: a practical and powerful approach to multiple testing. J R Stat Soc Ser B (Methodol) 57(1):289–300

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.